

Flowra – Master Installation Guide

Version: 3.0
Audience: Developers and IT administrators deploying Flowra for the first time.
Estimated time: 30–60 minutes for a complete local + Vercel deployment.

Table of Contents

- 1. [Prerequisites](#)
- 2. [Step 1 – Create Supabase Project](#)
- 3. [Step 2 – Run the Production SQL Installer](#)
- 4. [Step 3 – Configure Supabase Auth](#)
- 5. [Step 4 – Clone the Repository and Configure Environment](#)
- 6. [Step 5 – Local Development Test](#)
- 7. [Step 6 – Deploy to Vercel](#)
- 8. [Step 7 – Create the First Admin User](#)
- 9. [Step 8 – First-Time Company Setup](#)
- 10. [Verification Checklist](#)
- 11. [SQL File Map](#)

1. Prerequisites

Before you begin, make sure you have the following installed and available:

Requirement	Version	Notes
Node.js	18.x or 20.x LTS	nodejs.org
npm	9+ (ships with Node 18+)	<code>npm -v</code> to check
Git	Any recent version	<code>git --version</code> to check
Supabase account	Free tier is sufficient	supabase.com
Vercel account	Free tier is sufficient for production	vercel.com

Optional but recommended:

Tool	Purpose
Resend account	Email alerts and notification digest
Anthropic API key	AI-powered situation summaries (rule-based fallback is always active)

2. Step 1 — Create Supabase Project

1. Go to supabase.com and sign in.
2. Click **New Project**.
3. Fill in the form:
 - **Organization:** your organization
 - **Project name:** `flowra-production` (or whatever you prefer)
 - **Database password:** choose a strong password and **save it** — you will not see it again
 - **Region:** choose the region closest to your users. For Turkey, `eu-central-1` (Frankfurt) is the best available option.
4. Click **Create new project** and wait ~2 minutes for provisioning.

Once the project is ready, navigate to **Project Settings** → **API** and note down:

Value	Where to find it
Project URL	<code>https://<ref>.supabase.co</code>
anon / public key	Under "Project API keys"
service_role key	Under "Project API keys" — marked "secret"

Keep the service_role key private. It bypasses Row Level Security and must never be exposed in browser code.

3. Step 2 — Run the Production SQL Installer

The entire Flowra database schema — tables, functions, RLS policies, triggers, indexes, and seed reference data — is contained in a single canonical file:

```
supabase/FLOWRA_PRODUCTION_INSTALL.sql
```

How to run it

1. In the Supabase Dashboard, open your project.
2. Click **SQL Editor** in the left sidebar.
3. Click **New query**.
4. Open `supabase/FLOWRA_PRODUCTION_INSTALL.sql` from the cloned repository in a text editor, select all content, and paste it into the SQL Editor.
5. Click **Run** (or press `Cmd+Enter` / `Ctrl+Enter`).

The script takes 10–30 seconds. A successful run produces no errors — only informational `NOTICE` messages and a final confirmation message.

What the installer creates

- All 60+ application tables (companies, users, sales, expenses, journal entries, etc.)
- Row Level Security (RLS) policies for multi-tenant isolation
- Database functions for GL journal entry automation
- Triggers for automatic KDV calculation, interest accrual, audit log hashing
- MSUGT chart of accounts reference data (accounts 100–780)
- Default KDV rate configuration (1%, 10%, 18%)
- Reference data for Turkish corporate tax rates

Troubleshooting the SQL run

"already exists" errors: The SQL uses `CREATE TABLE IF NOT EXISTS` and `CREATE OR REPLACE FUNCTION` throughout. Duplicate errors should not appear on a fresh database. If you see them, the project may not be empty — consider creating a fresh Supabase project.

"permission denied" errors: Ensure you are running in the SQL Editor as the `postgres` superuser. The SQL Editor in the Supabase dashboard always runs as `postgres`.

Partial run: If the browser times out mid-run, check which tables exist (`SELECT tablename FROM pg_tables WHERE schemaname='public'`). If fewer than 50 tables exist, run `supabase/FLOWRA_PRODUCTION_UPGRADE.sql` to complete the installation safely (it is idempotent).

4. Step 3 — Configure Supabase Auth

Enable Email Authentication

1. In the Supabase Dashboard, go to **Authentication** → **Providers**.
2. Ensure **Email** is enabled.
3. Under Email settings:

- Set **Confirm email** to **disabled** for initial testing (re-enable before launch if your policy requires it).
- Leave all other defaults.

Configure Auth Redirect URLs

1. Go to **Authentication** → **URL Configuration**.
2. Set **Site URL** to your application URL:
 - For local development: `http://localhost:3000`
 - For production: `https://your-app.vercel.app` (update after Vercel deployment)
3. Under **Redirect URLs**, add all four entries:

```
http://localhost:3000/auth/callback
http://localhost:3000/**
https://your-app.vercel.app/auth/callback
https://your-app.vercel.app/**
```

Replace `your-app.vercel.app` with your actual Vercel domain once you have it. If you use a custom domain, add those entries as well.

5. Step 4 — Clone the Repository and Configure Environment

Clone

```
git clone <your-repo-url> flowra
cd flowra
npm install
```

Configure .env.local

Copy the example file:

```
cp .env.example .env.local
```

Open `.env.local` and fill in the required values:

```
# Required — from Supabase Dashboard → Project Settings → API
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# Required — application URL
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_SITE_URL=http://localhost:3000

# Required for cron authentication (any random string works locally)
CRON_SECRET=local-dev-secret

# Optional — email features
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
RESEND_FROM_EMAIL=flowra@yourcompany.com
ADMIN_DIGEST_EMAIL=admin@yourcompany.com

# Optional — AI situation summaries
ANTHROPIC_API_KEY=sk-ant-api03-XXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# Optional — log verbosity
LOG_LEVEL=debug
```

Important: `SUPABASE_SERVICE_ROLE_KEY` is a secret. Do not commit `.env.local` to version control. It is already listed in `.gitignore`.

6. Step 5 — Local Development Test

Start the development server:

```
npm run dev
```

Visit `http://localhost:3000`. You should see the Flowra login screen.

First login

There are no users yet. You must create the first user through Supabase Auth directly (see Step 7 below). After creating the user, log in and you will be prompted to create or join a company.

Smoke tests

Once logged in:

1. Navigate to `/dashboard` — should load the Komuta (CEO Cockpit) view.
2. Navigate to `/dashboard/finance` — should show the Finance Center with 8 tabs.
3. Navigate to `/dashboard/admin` — should show the Admin hub.

4. Open the browser console — no errors should appear.
5. Check the API health endpoint: `http://localhost:3000/api/health` — should return `{"status": "ok"}`.

7. Step 6 — Deploy to Vercel

Create the Vercel project

1. Push your code to a GitHub/GitLab/Bitbucket repository.
2. Go to vercel.com and click **Add New Project**.
3. Import your repository.
4. Framework detection should auto-select **Next.js**. Confirm:
 - **Framework Preset:** Next.js
 - **Root Directory:** `.` (the repo root)
 - **Build Command:** `npm run build` (default)
 - **Output Directory:** `.next` (default)

Set environment variables

Before the first deploy, go to the **Environment Variables** section in the Vercel project settings and add all required variables. Do not use `.env.local` values for production — generate new secrets.

Variable	Value	Notes
<code>NEXT_PUBLIC_SUPABASE_URL</code>	<code>https://your-ref.supabase.co</code>	From Supabase dashboard
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	anon key	From Supabase dashboard
<code>SUPABASE_SERVICE_ROLE_KEY</code>	service_role key	Mark as sensitive
<code>NEXT_PUBLIC_APP_URL</code>	<code>https://your-app.vercel.app</code>	Update after first deploy
<code>NEXT_PUBLIC_SITE_URL</code>	<code>https://your-app.vercel.app</code>	Same as above
<code>CRON_SECRET</code>	<code>openssl rand -hex 32</code> output	32+ char random token
<code>RESEND_API_KEY</code>	from resend.com	Required for email features
<code>RESEND_FROM_EMAIL</code>	verified sender address	Must be verified in Resend
<code>ADMIN_DIGEST_EMAIL</code>	your admin email	Receives daily alert digest
<code>ANTHROPIC_API_KEY</code>	from console.anthropic.com	Optional

Variable	Value	Notes
<code>LOG_LEVEL</code>	<code>info</code>	Recommended for production

Set all environment variables for **Production**, **Preview**, and **Development** environments unless otherwise noted. The `SUPABASE_SERVICE_ROLE_KEY` should be set for Production and Preview only, never for any environment that is publicly accessible.

Deploy

Click **Deploy**. The first build takes 2–4 minutes. Once complete, note your deployment URL (e.g., `https://flowra-abcl23.vercel.app`).

Update redirect URLs

After the first deploy, go back to **Supabase** → **Authentication** → **URL Configuration** and add your production URL:

```
https://your-app.vercel.app/auth/callback
https://your-app.vercel.app/**
```

Then update `NEXT_PUBLIC_APP_URL` and `NEXT_PUBLIC_SITE_URL` in Vercel environment variables to use your production URL and redeploy.

8. Step 7 — Create the First Admin User

There is no public registration flow. Users are invited by administrators. The first admin must be created directly through Supabase.

Option A: Supabase Dashboard (recommended for first user)

1. Go to **Supabase Dashboard** → **Authentication** → **Users**.
2. Click **Add user** → **Create new user**.
3. Enter an email and password.
4. The user will be created with a confirmed email, ready to log in immediately.

Option B: Supabase Auth API

```
curl -X POST 'https://your-ref.supabase.co/auth/v1/admin/users' \
  -H "apikey: YOUR_SERVICE_ROLE_KEY" \
  -H "Authorization: Bearer YOUR_SERVICE_ROLE_KEY" \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@yourcompany.com","password":"SecurePassword123","email_confirm":true}'
```

Assign admin role in the database

After creating the user, note the user's UUID from the Supabase Auth Users table. The role assignment happens automatically when the user completes company onboarding (they become the company owner with `admin` role). No manual database step is needed.

9. Step 8 — First-Time Company Setup

1. Log in to Flowra with the admin credentials you just created.
 2. You will be redirected to the company setup wizard.
 3. Fill in:
 - **Company name** (required)
 - **Tax ID / VKN** (10-digit Turkish tax number)
 - **Address** (required for PDF documents)
 - **Industry** (used for benchmark comparisons)
 4. Click **Create Company**. Flowra creates:
 - A new row in the `companies` table
 - An `admin` role entry in `company_members` for your user
 - Default alert rule thresholds
 - A default chart of accounts based on MSUGT
 5. After setup, you land on the Komuta dashboard. The system is ready to use.
-

10. Verification Checklist

Run through this checklist after completing the installation:

- ☐ `GET /api/health` returns `{"status":"ok"}`
- ☐ Login with admin credentials succeeds
- ☐ Dashboard loads without console errors
- ☐ `/dashboard/finance` shows all 8 tabs
- ☐ `/dashboard/admin` shows the admin panel
- ☐ Creating a test sale via `/dashboard/commercial` works
- ☐ The sale appears in the Finance → General tab
- ☐ `/dashboard/cfo` shows the Trial Balance (should be balanced)
- ☐ `/api/cron/overdue-update` returns `{"ok":true}` when called with `Authorization: Bearer <CRON_SECRET>`
- ☐ Environment variable `SUPABASE_SERVICE_ROLE_KEY` is NOT visible in browser sources

- ☐ Supabase Auth redirect URLs include your production domain
- ☐ `ENABLE_SEED` and `ENABLE_RESET` are not set in production Vercel environment

11. SQL File Map

The `supabase/` directory contains multiple SQL files. This table clarifies which ones to use and when.

File	Purpose	When to use
<code>FLOWRA_PRODUCTION_INSTALL.sql</code>	Canonical fresh install. Creates all tables, RLS policies, functions, triggers, and reference data from scratch.	New empty database only.
<code>FLOWRA_PRODUCTION_UPGRADE.sql</code>	Canonical upgrade. Adds new columns, tables, and functions to an existing installation. Idempotent — safe to run multiple times.	Upgrading an existing installation.
<code>flowra_phase9c_backfill.sql</code>	Backfills GL journal entries for all historical sales, expenses, and collections. Required when advancing from <code>shadow</code> to <code>parallel</code> GL mode.	Before activating <code>parallel</code> GL mode.
<code>flowra_phase9c_gl_primary_cutover.sql</code>	Advances the company's GL mode from <code>parallel</code> to <code>parallel</code>	After validating parallel GL mode.

File	Purpose	When to use
	<code>gl_primary</code> . Financial statements then read from the ledger, not from operational tables.	
<code>flowra_phase9c_rollback.sql</code>	Rolls back <code>gl_primary</code> to <code>parallel</code> . Safe recovery option.	If issues are found after <code>gl_primary</code> activation.
<code>db_audit.sql</code>	Schema health check. Reports missing tables, columns, and functions.	Diagnosing installation issues.
<code>schema_verify.sql</code>	Column-level verification. Checks all expected columns exist.	After upgrades, to confirm changes landed.
<code>repair_production.sql</code>	Emergency repair script. Fixes known schema corruption patterns.	Only if schema corruption is confirmed.
<code>supabase/archive/*</code>	Historical phase-by-phase migration scripts from development.	Reference only. Do not run.
<code>flowra_install.sql</code>	Legacy installer from an earlier version.	Superseded. Use <code>FLOWRA_PRODUCTION_INSTALL.sql</code> instead.

File	Purpose	When to use
<code>flowra_FULL_MIGRATION.sql</code>	Legacy upgrade script from an earlier version.	Superseded. Use <code>FLOWRA_PRODUCTION_UPGRADE.sql</code> instead.
<code>FLOWRA_FULL_INSTALL.sql</code>	Intermediate installer (pre-v3).	Superseded. Use <code>FLOWRA_PRODUCTION_INSTALL.sql</code> instead.
<code>governance.sql</code>	Standalone governance table creation.	Already included in <code>FLOWRA_PRODUCTION_INSTALL.sql</code> . Only needed if governance tables are missing from an old install.
<code>reconciliation_system.sql</code>	Standalone reconciliation table creation.	Already included in <code>FLOWRA_PRODUCTION_INSTALL.sql</code> . Only needed if reconciliation tables are missing from an old install.
<code>accounting_truth_v1.sql</code>	Establishes the GL accounting truth view.	Already included in <code>FLOWRA_PRODUCTION_INSTALL.sql</code> .
<code>grant-fix.sql</code>	Repairs missing <code>GRANT</code> permissions.	If API returns permission errors after upgrade.
<code>patch_company_settings_columns.sql</code>	Adds company settings columns.	Included in <code>FLOWRA_PRODUCTION_UPGRADE.sql</code> . Only run standalone if upgrade missed this.
<code>phase9_workflow_governance_patch.sql</code>	Adds workflow and governance columns.	Included in <code>FLOWRA_PRODUCTION_UPGRADE.sql</code> .
<code>FLOWRA_SYNC_PATCH.sql</code>	Sync patch for specific column additions.	Apply after <code>FLOWRA_PRODUCTION_UPGRADE.sql</code> if sync errors occur.

Rule of thumb:

- Fresh install → `FLOWRA_PRODUCTION_INSTALL.sql`
- Upgrading existing → `FLOWRA_PRODUCTION_UPGRADE.sql`
- Anything in `archive/` → read-only reference, never execute